

μOS++ III SMP

From the clone to the native test

µOS++ III SMP — from the clone to the native test

The commands used on 2026-10-08 to clone the five repositories from `github.com/dan-maio` into `/tmp/micro-os` and run the POSIX native tests with the system compiler (`native-cmake-sys`).

1. Create the folder and clone

```
mkdir -p /tmp/micro-os
cd /tmp/micro-os

git clone https://github.com/dan-maio/micro-os-plus-iii.git
git clone https://github.com/dan-maio/micro-os-plus-iii-aarch32.git
git clone https://github.com/dan-maio/micro-os-plus-iii-aarch64.git
git clone https://github.com/dan-maio/micro-os-plus-iii-posix-arch.git
git clone https://github.com/dan-maio/micro-os-plus-iii-cortexm.git
```

Each clone checks out its GitHub default branch, `smp`:

Repository	Branch	Commit
<code>micro-os-plus-iii</code>	<code>smp</code>	<code>38076f09</code> docs(tests): add new-tests-diff.md and pdf comparing xpack-development ...
<code>micro-os-plus-iii-aarch32</code>	<code>smp</code>	<code>e721dea</code> test(luckfox-lyra): smp_test_int4
<code>micro-os-plus-iii-aarch64</code>	<code>smp</code>	<code>8151c26</code> test(rpi3b): usb_test
<code>micro-os-plus-iii-posix-arch</code>	<code>smp</code>	<code>4ed05ad</code> chore: ignore every build* directory and the build products
<code>micro-os-plus-iii-cortexm</code>	<code>smp</code>	<code>71e98ff</code> docs: README for the six boards and the generic QEMU cores

To check what was cloned:

```
cd /tmp/micro-os
for r in micro-os-plus-iii micro-os-plus-iii-aarch32 micro-os-plus-iii-aarch64 \
    micro-os-plus-iii-posix-arch micro-os-plus-iii-cortexm; do
    echo "$r: [$(git -C $r branch --show-current)] $(git -C $r log -1 --format='%h %s')"
```

```
done
```

2. Install (xpack-dev-smp.md §13.3)

The native tests need the POSIX-arch port, so only that port is installed and linked; then the kernel test harness installs its dependencies.

```
cd /tmp/micro-os/micro-os-plus-iii-posix-arch
xpm install
xpm link

cd /tmp/micro-os/micro-os-plus-iii/tests
xpm run install-all
```

Result: both returned 0.

The kernel harness finds the ports as siblings of `micro-os-plus-iii` (`tests/cmake/tests-main.cmake`), which is why all five clones sit side by side in `/tmp/micro-os`.

3. Run the native tests (system compiler)

```
cd /tmp/micro-os/micro-os-plus-iii/tests
xpm run test-native-cmake-sys
```

`test-native-cmake-sys` runs, for the `debug` and then the `release` configuration:

```
xpm run prepare --config native-cmake-sys-debug
xpm run build   --config native-cmake-sys-debug
xpm run test    --config native-cmake-sys-debug
xpm run prepare --config native-cmake-sys-release
xpm run build   --config native-cmake-sys-release
xpm run test    --config native-cmake-sys-release
```

The compiler is the host's (`gcc (GCC) 16.2.1`); `test` runs `ctest -V` in `build/<config>`.

4. Results

Configuration	Result
<code>native-cmake-sys-debug</code>	16/16 passed
<code>native-cmake-sys-release</code>	16/16 passed

`xpm run test-native-cmake-sys` returned 0.

The 16 tests: flatfs-test, mutex-ceiling-test, mutex-stress, rtos-apis, smp-mat-test, smp-mutex-stress, smp-num-test, smp-pipeline-test, smp-pro-cons-test, smp-rtos-apis, smp_test0 ... smp_test4, cmsis-os-validator (each `native-<name>-host`).

To re-run a single test:

```
cd /tmp/micro-os/micro-os-plus-iii/tests/build/native-cmake-sys-debug
ctest -R '^native-smp-pipeline-test-host$' --output-on-failure
```

5. Possible correction: `env` must be `/usr/bin/env`

`micro-os-plus-iii/test_smpl/run-host.sh` starts each test binary through `env`, found in `PATH`:

```
timeout "$tmo" env "${env[@]}" "$exe" 2>&1 | tee "$log"      # line 78
timeout "$tmo" env "${env[@]}" "$exe" > "$log" 2>&1         # line 81
```

On some systems `PATH` puts `~/local/bin` before `/usr/bin`, and `~/local/bin` holds another program named `env`. That one runs instead of `/usr/bin/env`, and the test binaries are not started: every port test (`native-*-host`) prints nothing, ends in 0.02 s and is reported as `NO RESULT (rc=0)`, while `native-cmsis-os-validator-host` (started directly by `ctest`) passes.

Check which `env` is used:

```
type -a env
```

The `env` used must be `/usr/bin/env`. The correction, in `test_smpl/run-host.sh`, is to call it by its full path on both lines:

```
timeout "$tmo" /usr/bin/env "${env[@]}" "$exe" 2>&1 | tee "$log"
timeout "$tmo" /usr/bin/env "${env[@]}" "$exe" > "$log" 2>&1
```